JavaScript is disabled on your browser.

compact1, compact2, compact3

java.text

Class DecimalFormatSymbols

java.lang.Object
    java.text.DecimalFormatSymbols

All Implemented Interfaces:

Serializable, Cloneable

```
public class DecimalFormatSymbols
extends Object
implements Cloneable, Serializable
```

This class represents the set of symbols (such as the decimal separator, the grouping separator, and so on) needed by `DecimalFormat` to format numbers. `DecimalFormat` creates for itself an instance of `DecimalFormatSymbols` from its locale data. If you need to change any of these symbols, you can get the `DecimalFormatSymbols` object from your `DecimalFormat` and modify it.

See Also:

Locale, DecimalFormat, Serialized Form

Constructor Summary

Constructors

Constructor	Description
DecimalFormatSymbols()	Create a DecimalFormatSymbols object for the default **FORMAT** locale.

DecimalFormatSymbols(**Locale** locale)	Create a DecimalFormatSymbols object for the given locale.

Method Summary

Concrete Methods

Modifier and Type	Method
Object	**clone**()
boolean	**equals**(**Object** obj)
static **Locale**[]	**getAvailableLocales**()
Currency	**getCurrency**()
String	**getCurrencySymbol**()
char	**getDecimalSeparator**()
char	**getDigit**()
String	**getExponentSeparator**()
char	**getGroupingSeparator**()
String	**getInfinity**()
static **DecimalFormatSymbols**	**getInstance**()

static **DecimalFormatSymbols**	**getInstance**(**Locale** locale)
String	**getInternationalCurrencySymbol**()
char	**getMinusSign**()
char	**getMonetaryDecimalSeparator**()
String	**getNaN**()
char	**getPatternSeparator**()
char	**getPercent**()
char	**getPerMill**()
char	**getZeroDigit**()
int	**hashCode**()
void	**setCurrency**(**Currency** currency)
void	**setCurrencySymbol**(**String** currency)
void	**setDecimalSeparator**(char decimalSeparator)

void	**setDigit**(char digit)
void	**setExponentSeparator**(**String** exp)
void	**setGroupingSeparator**(char groupingSeparator)
void	**setInfinity**(**String** infinity)
void	**setInternationalCurrencySymbol**(**String** currencyCode)
void	**setMinusSign**(char minusSign)
void	**setMonetaryDecimalSeparator**(char sep)
void	**setNaN**(**String** NaN)
void	**setPatternSeparator**(char patternSeparator)
void	**setPercent**(char percent)
void	**setPerMill**(char perMill)
void	**setZeroDigit**(char zeroDigit)

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

DecimalFormatSymbols

```
public DecimalFormatSymbols()
```

Create a DecimalFormatSymbols object for the default `FORMAT` locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed `DecimalFormatSymbolsProvider` implementations. For full locale coverage, use the `getInstance` method.

This is equivalent to calling `DecimalFormatSymbols(Locale.getDefault(Locale.Category.FORMAT))`.

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

DecimalFormatSymbols

```
public DecimalFormatSymbols(Locale locale)
```

Create a DecimalFormatSymbols object for the given locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed `DecimalFormatSymbolsProvider` implementations. For full locale coverage, use the `getInstance` method. If the specified locale contains the `Locale.UNICODE_LOCALE_EXTENSION` for the numbering system, the instance is initialized with the specified numbering system if the JRE implementation supports it. For example,

```
NumberFormat.getNumberInstance(Locale.forLanguageTag("th-TH-u-nu-thai"))
```

This may return a `NumberFormat` instance with the Thai numbering system, instead of the Latin numbering system.

Parameters:

`locale - the desired locale`

Throws:

`NullPointerException - if locale is null`

Method Detail

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `getInstance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `DecimalFormatSymbolsProvider` implementations. It must contain at least a `Locale` instance equal to `Locale.US`.

Returns:

an array of locales for which localized DecimalFormatSymbols instances are available.

Since:

1.6

getInstance

```
public static final DecimalFormatSymbols getInstance()
```

Gets the `DecimalFormatSymbols` instance for the default locale. This method provides access to `DecimalFormatSymbols` instances for locales supported by the Java runtime itself as well as for those supported by installed `DecimalFormatSymbolsProvider` implementations.

This is equivalent to calling `getInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a DecimalFormatSymbols instance.

Since:

1.6

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getInstance

```
public static final DecimalFormatSymbols getInstance(Locale locale)
```

Gets the `DecimalFormatSymbols` instance for the specified locale. This method provides access to `DecimalFormatSymbols` instances for locales supported by the Java runtime itself as well as for those supported by

installed DecimalFormatSymbolsProvider implementations. If the specified locale contains the Locale.UNICODE_LOCALE_EXTENSION for the numbering system, the instance is initialized with the specified numbering system if the JRE implementation supports it. For example,

```
NumberFormat.getNumberInstance(Locale.forLanguageTag("th-TH-u-nu-thai"))
```

This may return a NumberFormat instance with the Thai numbering system, instead of the Latin numbering system.

Parameters:

locale - the desired locale.

Returns:

a DecimalFormatSymbols instance.

Throws:

NullPointerException - if locale is null

Since:

1.6

getZeroDigit

```
public char getZeroDigit()
```

Gets the character used for zero. Different for Arabic, etc.

Returns:

the character used for zero

setZeroDigit

```
public void setZeroDigit(char zeroDigit)
```

Sets the character used for zero. Different for Arabic, etc.

Parameters:

zeroDigit - the character used for zero

getGroupingSeparator

```
public char getGroupingSeparator()
```

Gets the character used for thousands separator. Different for French, etc.

Returns:

the grouping separator

setGroupingSeparator

```
public void setGroupingSeparator(char groupingSeparator)
```

Sets the character used for thousands separator. Different for French, etc.

Parameters:

groupingSeparator - the grouping separator

getDecimalSeparator

```
public char getDecimalSeparator()
```

Gets the character used for decimal sign. Different for French, etc.

Returns:

the character used for decimal sign

setDecimalSeparator

```
public void setDecimalSeparator(char decimalSeparator)
```

Sets the character used for decimal sign. Different for French, etc.

Parameters:

decimalSeparator - the character used for decimal sign

getPerMill

```
public char getPerMill()
```

Gets the character used for per mille sign. Different for Arabic, etc.

Returns:

the character used for per mille sign

setPerMill

```
public void setPerMill(char perMill)
```

Sets the character used for per mille sign. Different for Arabic, etc.

Parameters:

`perMill - the character used for per mille sign`

getPercent

```
public char getPercent()
```

Gets the character used for percent sign. Different for Arabic, etc.

Returns:

`the character used for percent sign`

setPercent

```
public void setPercent(char percent)
```

Sets the character used for percent sign. Different for Arabic, etc.

Parameters:

`percent - the character used for percent sign`

getDigit

```
public char getDigit()
```

Gets the character used for a digit in a pattern.

Returns:

`the character used for a digit in a pattern`

setDigit

```
public void setDigit(char digit)
```

Sets the character used for a digit in a pattern.

Parameters:

`digit - the character used for a digit in a pattern`

getPatternSeparator

```
public char getPatternSeparator()
```

Gets the character used to separate positive and negative subpatterns in a pattern.

Returns:

`the pattern separator`

setPatternSeparator

```
public void setPatternSeparator(char patternSeparator)
```

Sets the character used to separate positive and negative subpatterns in a pattern.

Parameters:

`patternSeparator - the pattern separator`

getInfinity

```
public String getInfinity()
```

Gets the string used to represent infinity. Almost always left unchanged.

Returns:

`the string representing infinity`

setInfinity

```
public void setInfinity(String infinity)
```

Sets the string used to represent infinity. Almost always left unchanged.

Parameters:

`infinity - the string representing infinity`

getNaN

```
public String getNaN()
```

Gets the string used to represent "not a number". Almost always left unchanged.

Returns:

the string representing "not a number"

setNaN

```
public void setNaN(String NaN)
```

Sets the string used to represent "not a number". Almost always left unchanged.

Parameters:

`NaN - the string representing "not a number"`

getMinusSign

```
public char getMinusSign()
```

Gets the character used to represent minus sign. If no explicit negative format is specified, one is formed by prefixing minusSign to the positive format.

Returns:

`the character representing minus sign`

setMinusSign

```
public void setMinusSign(char minusSign)
```

Sets the character used to represent minus sign. If no explicit negative format is specified, one is formed by prefixing minusSign to the positive format.

Parameters:

`minusSign - the character representing minus sign`

getCurrencySymbol

```
public String getCurrencySymbol()
```

Returns the currency symbol for the currency of these DecimalFormatSymbols in their locale.

Returns:

the currency symbol

Since:

1.2

setCurrencySymbol

```
public void setCurrencySymbol(String currency)
```

Sets the currency symbol for the currency of these DecimalFormatSymbols in their locale.

Parameters:

currency - the currency symbol

Since:

1.2

getInternationalCurrencySymbol

```
public String getInternationalCurrencySymbol()
```

Returns the ISO 4217 currency code of the currency of these DecimalFormatSymbols.

Returns:

the currency code

Since:

1.2

setInternationalCurrencySymbol

```
public void setInternationalCurrencySymbol(String currencyCode)
```

Sets the ISO 4217 currency code of the currency of these DecimalFormatSymbols. If the currency code is valid (as defined by `Currency.getInstance`), this also sets the currency attribute to the corresponding Currency instance and the currency symbol attribute to the

currency's symbol in the DecimalFormatSymbols' locale. If the currency code is not valid, then the currency attribute is set to null and the currency symbol attribute is not modified.

Parameters:

currencyCode - the currency code

Since:

1.2

See Also:

setCurrency(java.util.Currency), setCurrencySymbol(java.lang.String)

getCurrency

```
public Currency getCurrency()
```

Gets the currency of these DecimalFormatSymbols. May be null if the currency symbol attribute was previously set to a value that's not a valid ISO 4217 currency code.

Returns:

the currency used, or null

Since:

1.4

setCurrency

```
public void setCurrency(Currency currency)
```

Sets the currency of these DecimalFormatSymbols. This also sets the currency symbol attribute to the currency's symbol in the DecimalFormatSymbols' locale, and the international currency symbol attribute to the currency's ISO 4217 currency code.

Parameters:

currency - the new currency to be used

Throws:

NullPointerException - if currency is null

Since:

1.4

See Also:

setCurrencySymbol(java.lang.String), setInternationalCurrencySymbol(java.lang.String)

getMonetaryDecimalSeparator

```
public char getMonetaryDecimalSeparator()
```

Returns the monetary decimal separator.

Returns:

the monetary decimal separator

Since:

1.2

setMonetaryDecimalSeparator

```
public void setMonetaryDecimalSeparator(char sep)
```

Sets the monetary decimal separator.

Parameters:

sep - the monetary decimal separator

Since:

1.2

getExponentSeparator

```
public String getExponentSeparator()
```

Returns the string used to separate the mantissa from the exponent. Examples: "x10^" for 1.23x10^4, "E" for 1.23E4.

Returns:

the exponent separator string

Since:

1.6

See Also:

setExponentSeparator(java.lang.String)

setExponentSeparator

```
public void setExponentSeparator(String exp)
```

Sets the string used to separate the mantissa from the exponent. Examples: "x10^" for 1.23x10^4, "E" for 1.23E4.

Parameters:

exp - the exponent separator string

Throws:

NullPointerException - if exp is null

Since:

1.6

See Also:

getExponentSeparator()

clone

public Object clone()

Standard override.

Overrides:

clone in class Object

Returns:

a clone of this instance.

See Also:

Cloneable

equals

public boolean equals(Object obj)

Override equals.

Overrides:

equals in class Object

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is the same as the obj argument; false otherwise.

See Also:

Object.hashCode(), HashMap

hashCode

public int hashCode()

Override hashCode.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.